AI ASSISTED CODING

ASSIGNMENT-8

NAME: Varshitha. G

HTNO: 2303A51103

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username (username) and then implement the function using Test-Driven Development principles.

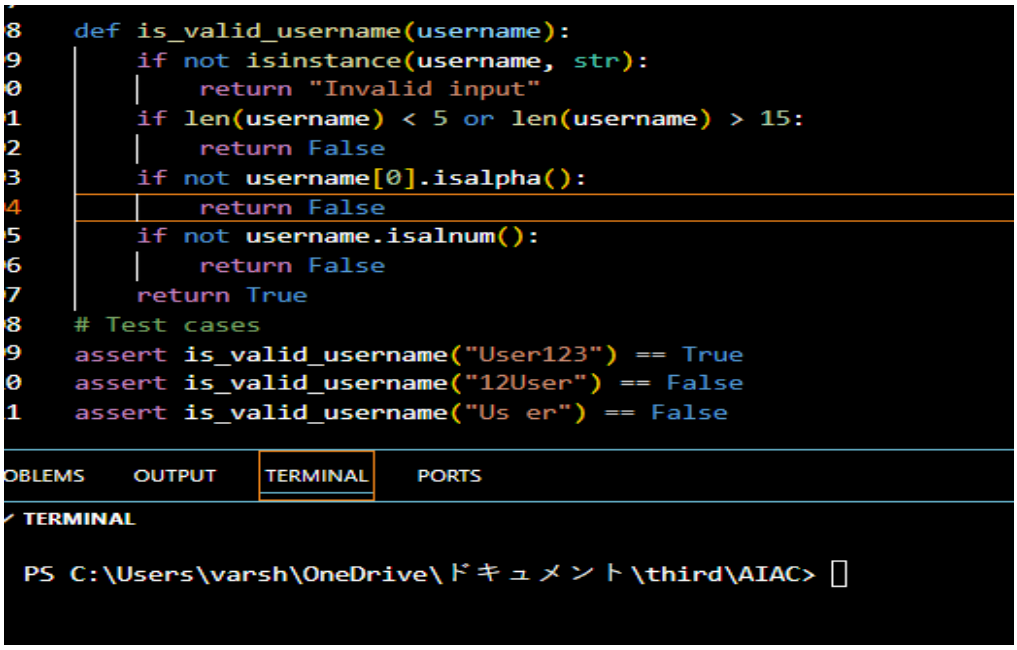
• Requirements:

o Username length must be between 5 and 15 characters.

o Must contain only alphabets and digits.

o Must not start with a digit.

o No spaces allowed.



Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.

• Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

assert classify_value("abc") == "Invalid Input"

Expected Output #2:

• Function correctly classifying values and passing all test cases.

```
113  def classify_value(x):
114      if x==0:
115          return "Zero"
116      if isinstance(x, (int, float)):
117          if x%2==0:
118              return "Even"
119          else:
120              return "Odd"
121      return "Invalid Input"
122  # Test cases
123  assert classify_value(8) == "Even"
124  assert classify_value(7) == "Odd"
125  assert classify_value("abc") == "Invalid Input"
```

```
PROBLEMS   OUTPUT   TERMINAL   PORTS

TERMINAL

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/
ppData/Local/Programs/Python/Python313/python.exe c:/Users/vars
ive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

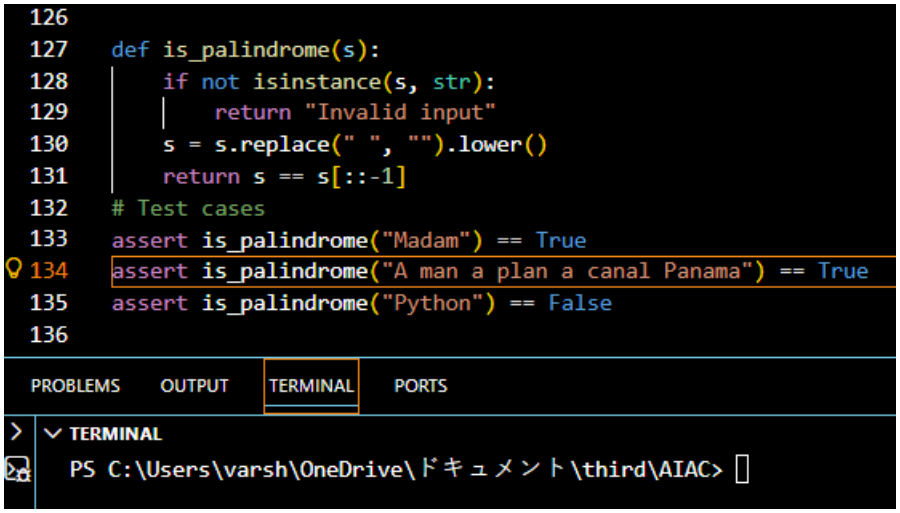
assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") == True

assert is_palindrome("Python") == False

Expected Output #3:

• Function correctly identifying palindromes and passing all AI-generated tests.



Task Description #4 (Email ID Validation – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

• Requirements:

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly.

Task 5 (Perfect Number Checker – Test Case Design)

• Function: Check if a number is a perfect number (sum of divisors = number).

• Test Cases to Design:

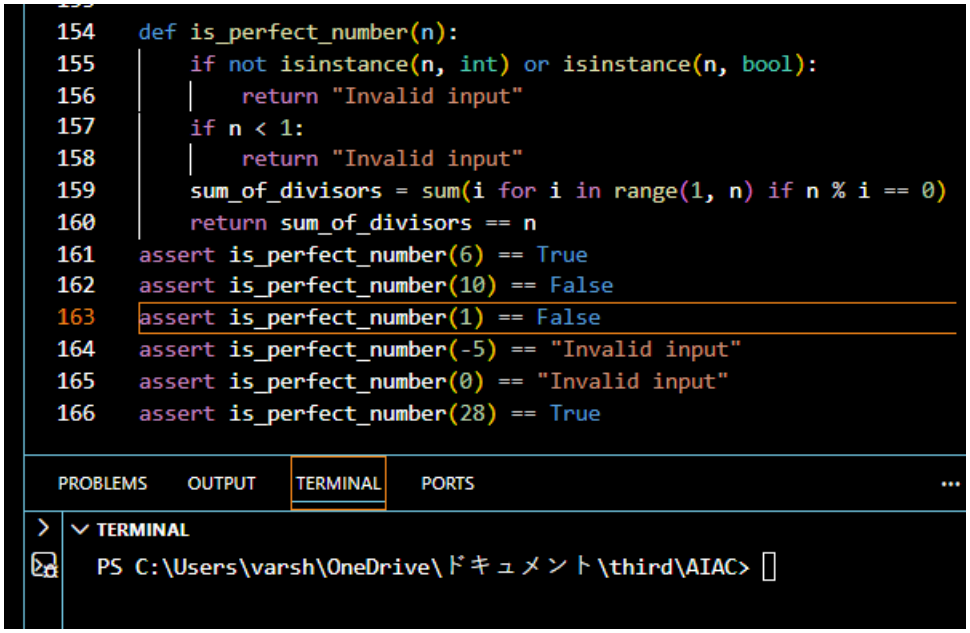
o Normal case: 6 → True, 10 → False.

o Edge case: 1.

o Negative number case.

o Larger case: 28.

• Requirement: Validate correctness with assertions.



Task 6 (Abundant Number Checker – Test Case Design)

• Function: Check if a number is abundant (sum of divisors >

number).

• Test Cases to Design:

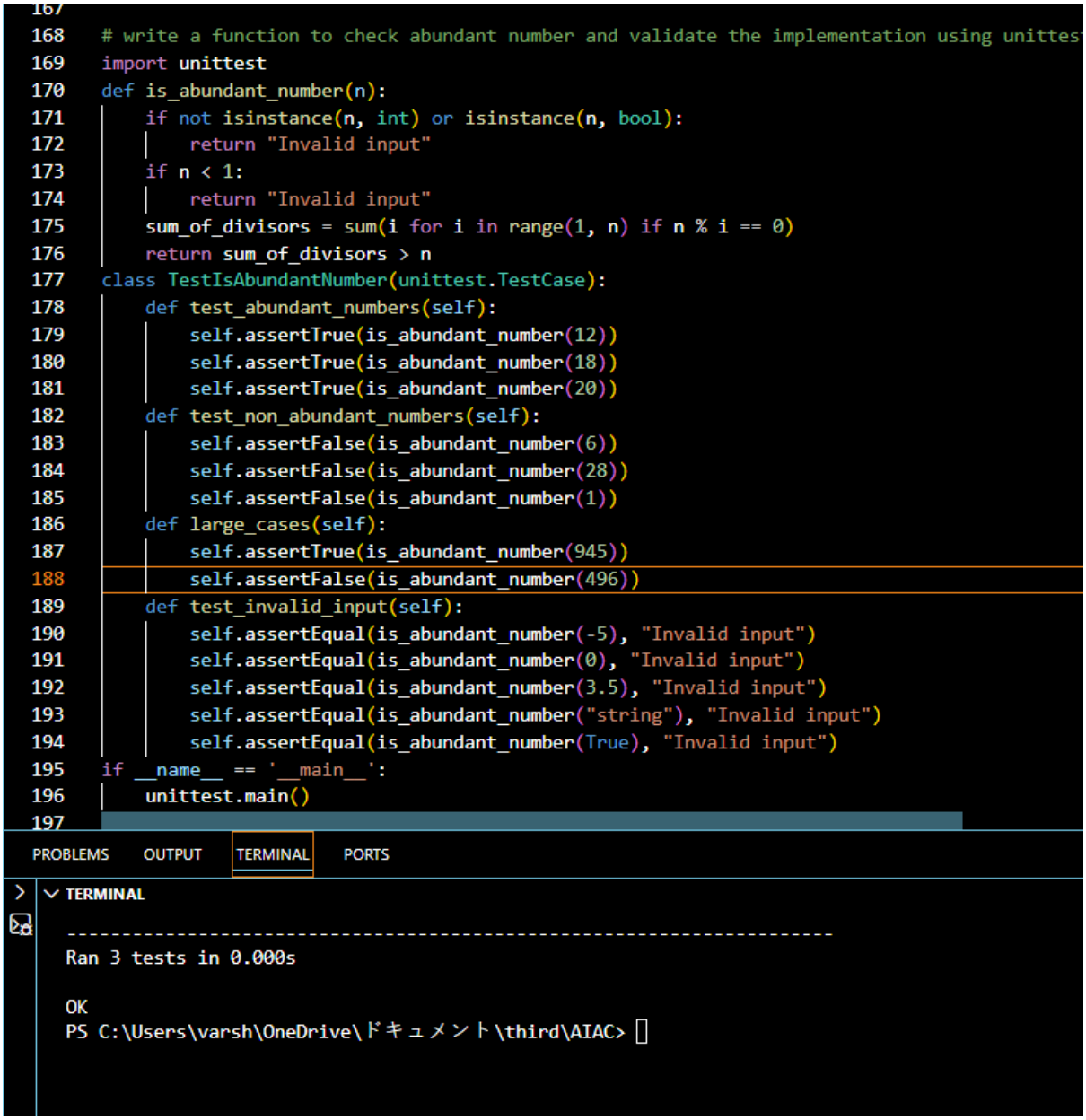
o Normal case: 12 → True, 15 → False.

o Edge case: 1.

o Negative number case.

o Large case: 945.

Requirement: Validate correctness with unittest



Task 7 (Deficient Number Checker – Test Case Design)

• Function: Check if a number is deficient (sum of divisors < number).

• Test Cases to Design:

o Normal case: 8 → True, 12 → False.

o Edge case: 1.

o Negative number case.

o Large case: 546.

Requirement: Validate correctness with pytest.

```python
# write a function to check if a number is deficient (sum of divisors is less than the numb
def is_deficient_number(n):
    if not isinstance(n, int) or isinstance(n, bool):
        return "Invalid input"
    if n < 1:
        return "Invalid input"
    sum_of_divisors = sum(i for i in range(1, n) if n % i == 0)
    return sum_of_divisors < n
def test_is_deficient_number():
    assert is_deficient_number(8) == True
    assert is_deficient_number(12) == True
    assert is_deficient_number(1) == True
    assert is_deficient_number(546) == False
    assert is_deficient_number(28) == False
    assert is_deficient_number(-5) == "Invalid input"
    assert is_deficient_number(0) == "Invalid input"
    assert is_deficient_number(3.5) == "Invalid input"
    assert is_deficient_number("string") == "Invalid input"
    assert is_deficient_number(True) == "Invalid input"
```

```
BLEMS  OUTPUT  TERMINAL  PORTS
TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/
ython313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 8 :

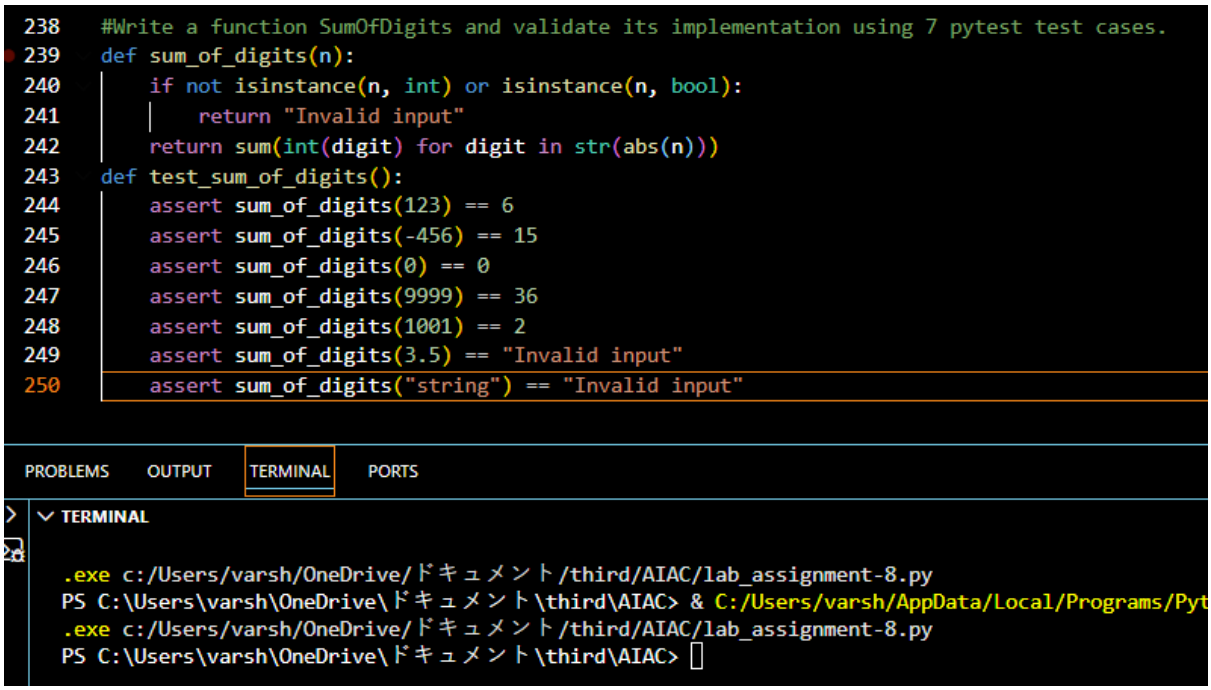
Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

```python
# Write a function LeapYearChecker and validate its implementation using 10 pytest test cases
def is_leap_year(year):
    if not isinstance(year, int) or isinstance(year, bool):
        return "Invalid input"
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    return False
def test_is_leap_year():
    assert is_leap_year(2020) == True
    assert is_leap_year(1900) == False
    assert is_leap_year(2000) == True
    assert is_leap_year(2021) == False
    assert is_leap_year(2400) == True
    assert is_leap_year(-4) == "Invalid input"
    assert is_leap_year(0) == "Invalid input"
    assert is_leap_year(3.5) == "Invalid input"
    assert is_leap_year("string") == "Invalid input"
    assert is_leap_year(True) == "Invalid input"
```

```
ROBLEMS  OUTPUT  TERMINAL  PORTS
TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C …
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Pyth
.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
```

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.



```
#Write a function SumOfDigits and validate its implementation using 7 pytest test cases.
def sum_of_digits(n):
    if not isinstance(n, int) or isinstance(n, bool):
        return "Invalid input"
    return sum(int(digit) for digit in str(abs(n)))
def test_sum_of_digits():
    assert sum_of_digits(123) == 6
    assert sum_of_digits(-456) == 15
    assert sum_of_digits(0) == 0
    assert sum_of_digits(9999) == 36
    assert sum_of_digits(1001) == 2
    assert sum_of_digits(3.5) == "Invalid input"
    assert sum_of_digits("string") == "Invalid input"
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS
TERMINAL
.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Pyt
.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```python
# Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.
def bubble_sort(arr):
    if not isinstance(arr, list) or any(not isinstance(x, (int, float)) for x in arr):
        return "Invalid input"
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
def test_bubble_sort():
    assert bubble_sort([64, 34, 25, 12, 22, 11, 90]) == [11, 12, 22, 25, 34, 64, 90]
    assert bubble_sort([5, 1, 4, 2, 8]) == [1, 2, 4, 5, 8]
    assert bubble_sort([]) == []
    assert bubble_sort([1]) == [1]
    assert bubble_sort([3.5, 2.1, 4.8]) == [2.1, 3.5, 4.8]
    assert bubble_sort([-5, -2, -3]) == [-5, -3, -2]
    assert bubble_sort([0]) == [0]
    assert bubble_sort([1.5]) == [1.5]
    assert bubble_sort([-1.5]) == [-1.5]
    assert bubble_sort([3.5]) == [3.5]
    assert bubble_sort([-3.5]) == [-3.5]
    assert bubble_sort([0.0]) == [0.0]
    assert bubble_sort([-0.0]) == [-0.0]
    assert bubble_sort([1e10, -1e10]) == [-1e10, 1e10]
    assert bubble_sort([-1e10, -1e10]) == [-1e10, -1e10]
    assert bubble_sort([1e10, 1e10]) == [1e10, 1e10]
    assert bubble_sort([3.14, 2.71]) == [2.71, 3.14]
    assert bubble_sort([-3.14, -2.71]) == [-3.14, -2.71]
    assert bubble_sort([0.001, -0.001]) == [-0.001, 0.001]
    assert bubble_sort([-0.001, -0.001]) == [-0.001, -0.001]
    assert bubble_sort([1000000]) == [1000000]
    assert bubble_sort([-1000000]) == [-1000000]
    assert bubble_sort([9999999999]) == [9999999999]
```

```
ROBLEMS  OUTPUT  TERMINAL  PORTS
TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.ex
rsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases

```python
# Write a function ReverseString and validate its implementation using 5 unittest test cases
def reverse_string(s):
    if not isinstance(s, str):
        return "Invalid input"
    return s[::-1]
import unittest
class TestReverseString(unittest.TestCase):
    def test_reverse_string(self):
        self.assertEqual(reverse_string("hello"), "olleh")
        self.assertEqual(reverse_string("Python"), "nohtyP")
        self.assertEqual(reverse_string(""), "")
        self.assertEqual(reverse_string("A man a plan a canal Panama"), "amanaP lanac a nalp a nam A")
        self.assertEqual(reverse_string(123), "Invalid input")
if __name__ == '__main__':
    unittest.main()
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS
TERMINAL
rsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest testcases.

```python
# Write a function AnagramChecker and validate its implementation using 10 unittest test cases.
def are_anagrams(str1, str2):
    if not isinstance(str1, str) or not isinstance(str2, str):
        return "Invalid input"
    return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
import unittest
class TestAnagramChecker(unittest.TestCase):
    def test_anagrams(self):
        self.assertTrue(are_anagrams("listen", "silent"))
        self.assertTrue(are_anagrams("Triangle", "Integral"))
    def test_non_anagrams(self):
        self.assertFalse(are_anagrams("hello", "world"))
        self.assertFalse(are_anagrams("Python", "Java"))
    def test_invalid_input(self):
        self.assertEqual(are_anagrams(123, "abc"), "Invalid input")
        self.assertEqual(are_anagrams("abc", 123), "Invalid input")
        self.assertEqual(are_anagrams(True, "abc"), "Invalid input")
        self.assertEqual(are_anagrams("abc", False), "Invalid input")
    def test_edge_cases(self):
        self.assertTrue(are_anagrams("", ""))
        self.assertTrue(are_anagrams("a", "a"))
    def test_case_sensitivity(self):
        self.assertTrue(are_anagrams("Listen", "Silent"))
        self.assertTrue(are_anagrams("Triangle", "Integral"))
    def test_whitespace_handling(self):
        self.assertTrue(are_anagrams("Dormitory", "Dirty Room"))
        self.assertTrue(are_anagrams("Conversation", "Voices Rant On"))
    def test_special_characters(self):
        self.assertTrue(are_anagrams("A!B@C#", "C#B@A!"))
        self.assertFalse(are_anagrams("A!B@C#", "C#B@D!"))
    def test_numeric_strings(self):
        self.assertTrue(are_anagrams("123", "321"))
        self.assertFalse(are_anagrams("123", "322"))
    def test_mixed_input(self):
        self.assertEqual(are_anagrams("123", 321), "Invalid input")
        self.assertEqual(are_anagrams(123, "321"), "Invalid input")
if __name__ == '__main__':
    unittest.main()
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

TERMINAL
----------------------------------------------------------------------
Ran 9 tests in 0.007s

OK
```

Task 13 :

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

```python
# Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.
def is_armstrong_number(n):
    if not isinstance(n, int) or isinstance(n, bool):
        return "Invalid input"
    num_str = str(abs(n))
    num_digits = len(num_str)
    armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
    if armstrong_sum == abs(n):
        return True
    return False
import unittest
class TestArmstrongChecker(unittest.TestCase):
    def test_armstrong_numbers(self):
        self.assertTrue(is_armstrong_number(153))
        self.assertTrue(is_armstrong_number(0))
        self.assertTrue(is_armstrong_number(9474))
    def test_non_armstrong_numbers(self):
        self.assertFalse(is_armstrong_number(145))
        self.assertFalse(is_armstrong_number(10))
        self.assertFalse(is_armstrong_number(123))
    def test_invalid_input(self):
        self.assertEqual(is_armstrong_number(-153), "Invalid input")
        self.assertEqual(is_armstrong_number(3.5), "Invalid input")
        self.assertEqual(is_armstrong_number("string"), "Invalid input")
        self.assertEqual(is_armstrong_number(True), "Invalid input")
if __name__ == '__main__':
    unittest.main()
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python
Ran 3 tests in 0.001s

FAILED (failures=1)
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```